SquishBox

Release 1.3.0

Bill Peterson

Sep 08, 2026

CONTENTS:

The SquishBox is an add-on board for the Raspberry Pi 3B+/4 that bundles a DAC, 1/4" ouputs, MIDI in/out with mini-jacks, a 16x2 LCD, and a pushbutton rotary encoder. It is designed to be an embedded computer for audio applications, such as a synth module or audio player. It has a python API for easy access to buttons, LCD, and system functions. Several pre-built apps are included.

- Official Documentation
- Source Repository

The SquishBox can be fabricated from scratch using the included design files, or obtained as a kit or fully assembled device from the Geek Funk Labs store. Software is installed in an existing Raspberry Pi OS using a command-line installer script.

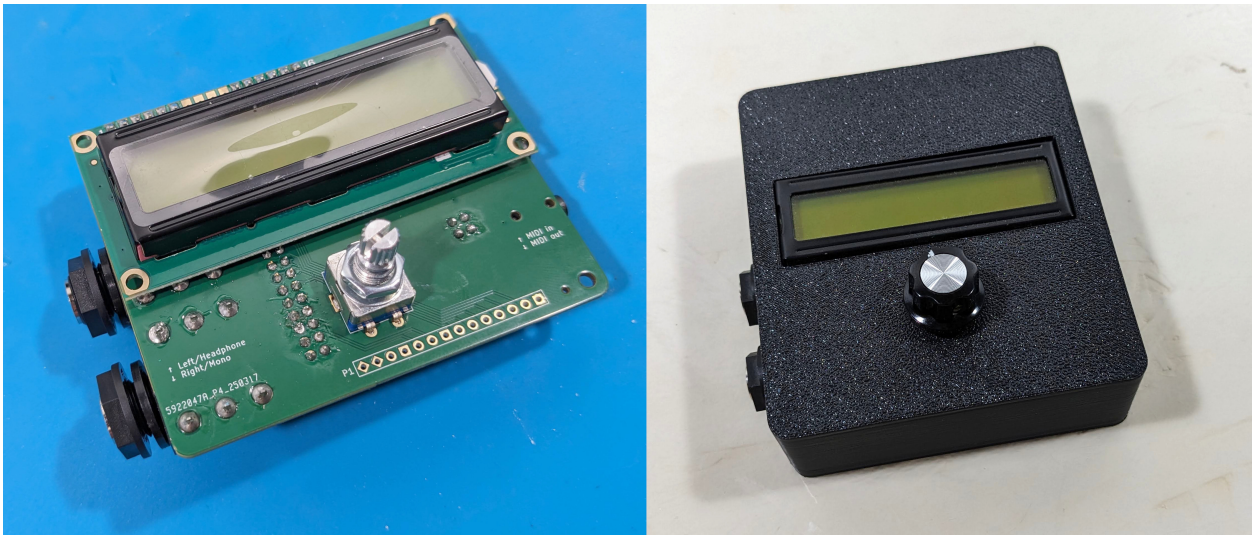


Fig. 1: Assembled SquishBox electronics (left) and enclosure (right)

CHAPTER

ONE

QUICK START

Follow the steps below to get a SquishBox up and running quickly. Some or all of the hardware steps can be skipped by purchasing a kit or fully assembled unit from the Geek Funk Labs store.

1. *Fabricating PCB and Enclosure*
2. *Putting it Together*
3. *Installing the Software*
4. *Basic Usage*

1.1 Fabricating PCB and Enclosure

The PCB fabrication files include only the surface-mount components for automated assembly. Through-hole components are intended to be sourced separately and soldered by hand.

Load the enclosure model files in `hardware/enclosure/` into your preferred slicer software for 3D printing. All parts are designed to print flat-side down without supports.

Three enclosure variants are provided:

- **basic** — Fits Raspberry Pi 3 Model B+ and Raspberry Pi 4 Model B. A Raspberry Pi 5 can fit, but not with the standard fan installed.
- **deluxe** — Fits Pi 3/4 and adds mounting points for two momentary footswitches and two LEDs, which can be connected to the P1 header on the SquishBox PCB.
- **mini** — Fits Raspberry Pi Model A+, with an enlarged cutout to accommodate wide USB cables or adapters.

Recommended print notes:

- PLA is sufficient, though other materials may be used for specific traits
- Infill is not critical due to the small internal volume
- Use at least 2–3 wall layers for good strength

1.2 Putting it Together

Insert the through-hole components from the side of the PCB with the matching silkscreen outlines, then solder them in place.

Install the LCD module last, as it blocks access to several other solder points.

Mount the completed PCB into the enclosure by passing the rotary encoder shaft and audio jacks through the corresponding openings, then secure them with the included washers and nuts.

Install the Raspberry Pi by plugging its GPIO header into the 2x20 socket, then fasten the lid in place.

1.3 Installing the Software

The SquishBox software is targeted for a Pi 3B+/4 running Raspberry Pi OS (64-bit, based on Debian 13 "Trixie") or newer.

Before installing it's advisable to perform an upgrade to make sure your audio stack is up to date:

```
sudo apt update
sudo apt upgrade
```

To run the install script, log in as a regular user and enter:

```
bash <(curl -sL geekfunklabs.com/squishbox)
```

Answer the prompts, wait for the installation to complete, then reboot the Pi to activate the LCD and control interface.

1.4 Basic Usage

On first boot, the SquishBox launches a menu system for starting apps or changing system settings.

Controls:

- Turn the rotary encoder to move through menu items
- Tap the encoder to confirm a selection
- Press and hold the encoder to cancel or return to the previous screen

To safely power down before disconnecting power, choose **Shutdown** from the **Exit** menu.

CHAPTER

TWO

USER GUIDE

When powered on, the SquishBox starts a launcher that allows you to run applications, browse tools, or modify system settings.

Navigation is designed to be simple and usable without a keyboard or display larger than the built-in LCD.

2.1 Basic Controls

The primary control is the pushbutton rotary encoder.

- Turn the encoder to move through menus or adjust values
- Tap the encoder to confirm a selection
- Press and hold the encoder to cancel, go back, or exit the current screen

Most built-in applications follow this same control scheme.

2.1.1 Text Entry

The SquishBox includes an on-screen text entry mode for naming files, editing settings, and entering text without a keyboard.

Pressing the encoder toggles between two cursor modes:

- **Blinking block cursor** — Move the cursor position by turning the encoder
- **Underline cursor** — Change the current character by turning the encoder

This allows text to be entered entirely from the front panel.

If a USB or wireless keyboard is connected to the Raspberry Pi, keyboard input is also accepted anywhere text entry is available.

2.1.2 Safe Shutdown

To safely power down the system before disconnecting power, choose **Shutdown** from the **Exit** menu.

This helps prevent filesystem corruption or SD card damage.

2.2 Software

2.2.1 File locations

By default, configuration and media files for the SquishBox are organized in the folder structure shown below. Directories are created by apps as needed.

```
~/Squishbox/
  ├── config/
  ├── banks/
  │    ├── fluidpatcher/
  │    └── amsynth/
  ├── media/
  │    ├── sounds/
  │    ├── midi/
  │    └── music/
  ├── playlists/
  └── sets/
```

The main configuration file is stored at `config/squishboxconf.yaml`, and defines settings such as:

- LCD configuration
- UI timings
- Inputs/Outputs and bindings
- Custom LCD characters

The `config` directory is also the default location for app configuration files.

2.2.2 User Interface

The SquishBox launcher starts automatically on boot and serves as the main entry point for applications and utilities. It provides a menu for launching programs and scripts, and also acts as a fallback environment if an application exits or crashes. Additional directories containing user scripts can be added using the `app_dirs` configuration option.

Most applications, including the launcher, provide access to shared system menus for configuring MIDI, Wi-Fi, LCD settings, and power options.

MIDI Routing

The SquishBox manages connections between MIDI devices and applications as defined in the `midi_connections` section of the configuration file.

```
midi_connections:
- DONNER DMK25Pro:0(DONNER DMK25Pro MIDI 1)>FluidPatcher:0(FluidPatcher)
- QuNexus:0(QuNexus Control Surface)>SquishBox minijacks:1(SquishBox minijacks Out)
```

These settings can be easily managed from the MIDI Settings menu by first selecting the MIDI input device, then selecting the destination.

Applications that use MIDI internally (such as `fpatcherbox`) typically appear as available destinations only while they are running.

The port `SquishBox MIDI out` sends messages from any hardware controls that are configured to do so (e.g. footswitches). The `SquishBox MIDI in` port is used by the system to detect when new devices are connected.

Special `any` entries may be used to create automatic routing rules. For example:

- Connect any MIDI device to a specific application
- Connect a specific MIDI device to any application
- Automatically connect any device to any available application (`any>any` - the default behavior)

Wi-Fi Settings

The Wi-Fi Settings menu allows scanning for wireless networks, connecting to access points, and enabling or disabling Wi-Fi entirely.

Most applications display a Wi-Fi status icon. This icon indicates whether the Wi-Fi hardware is enabled, not whether the system is currently connected to a network.

Disabling Wi-Fi may improve reliability in some situations, since Wi-Fi scanning and background network activity can increase memory usage and occasionally interfere with real-time audio applications.

Power Menu

The Power menu provides options to reboot or shut down the system, as well as a **Shell** option to exit the currently running application and return to the launcher.

If the system is rebooted or powered off while an application is running, the launcher automatically restarts that application on the next boot. This allows applications such as `amsynthbox` or `fpatcherbox` to behave like dedicated standalone instruments that automatically resume after power cycling.

2.2.3 Included Apps

The included applications serve two purposes:

- Fully supported tools and audio programs for daily use
- Reference examples for developers using the SquishBox Python API

Bug reports and feature requests can be submitted through the project GitHub issue tracker.

`fpatcherbox`

Flexible synth and sound module built around the FluidSynth engine using the FluidPatcher Python package.

The interface allows:

- Selecting patches, adding/removing/saving patches, and loading/saving banks
- Adjusting FluidSynth's built-in reverb and chorus effects
- Live selection of soundfont presets on any channel
- Creation of MIDI keyboard routing layers on-the-fly

Pressing and holding the encoder at the main screen toggles between displaying MIDI activity and full MIDI message information.

The app recognizes several custom router rule parameters:

`lcdwrite`
: Writes a message on the LCD when a MIDI event matches the rule. If the rule also has a `format` parameter of the form `w.n`, the event value is written with the specified width and number of decimal places.

`setpin`
: Sets the named SquishBox output. Can be used to turn an LED on/off in response to a MIDI event.

`patch`
: Applies a patch by name or increments the current patch by an amount.

amsynthbox

Front-end wrapper for the amsynth analog-modeling synthesizer.

All synth parameters can be adjusted using the menu, or bound to MIDI control change events using the **MIDI Learn** option. The `midi_channel` option in the configuration file sets the MIDI channel amsynth will respond to. A value of `0` responds to any channel.

trackbox

Playlist-based music player with live track reordering and quick cuts. Most types of audio files can be played, and are stored in `media/music/` by default. The root location for music is set using the `tracks_path` item in the configuration file.

The list of music files to play can be edited from the **Edit Tracklist** option in the menu. Track lists are saved as YAML files.

```
tracks_path: chill
first: 0
tracks:
- file: Hotel Pools - Melt.mp3
- file: downloadedfile1823.mp3
  name: Mallsoft Vibes
  start: 8:31
  end: 12:15
- file: synthwave_mixdown.wav
  name: My Chill Beats
  level: 0.8
```

Tracklists can also have a `tracks_path` parameter that is relative to the config file value or absolute. Tracklists start at the last played track, or the index given by `first` if present.

sbedit

Lightweight text editor that supports both encoder input and keyboards. Useful for editing config files or making changes to banks. When scrolling through lines, the line number is flashed at the left edge of the screen. Tapping the encoder edits the current line. Pressing and holding the encoder opens a menu for loading/saving files or inserting/deleting rows.

sbcommander

File manager for copying, moving, renaming, deleting files, and running shell commands. Able to mount/unmount USB drives and copy files, making it a useful tool for managing SquishBox content without a network connection.

Select files to act upon using the **Choose File(s)..** menu option. Canceling file selection chooses the current directory, allowing selection of multiple files or the entire directory tree.

Shell commands can be entered using encoder input or a keyboard. If file(s) are currently selected, they are added as arguments at the end of the command. The output of the command is displayed on the LCD and can be scrolled by turning the encoder.

glyphedit

Utility for creating and editing custom LCD glyph characters.

2.3 Hardware Connections

2.3.1 Audio Outputs

The SquishBox provides two 1/4” audio output jacks.

Physical orientation:

- **Rear jack** = Left / Headphone output
- **Front jack** = Right / Mono output

(When viewed face-on, these appear on the left and right sides respectively.)

Behavior:

- If only the **Right / Mono** jack is connected, left and right channels are summed to mono on that output.
- If only the **Left / Headphone** jack is connected, stereo audio is routed to the tip and sleeve of a TRS connector for headphone use.

The PCB silkscreen also labels both jacks.

Fig. 1: Audio ports in mono connection mode

2.3.2 MIDI TRS Ports

The SquishBox includes MIDI input and output on 3.5 mm TRS minijacks.

Physical orientation:

- **Front jack** = MIDI Out
- **Rear jack** = MIDI In

(When viewed face-on, MIDI In is on the right.)

The PCB silkscreen also identifies each port.

Headers with jumper blocks allow the MIDI jacks to be configured for either TRS MIDI wiring standard:

- **Type A** = horizontal jumper position (=)
- **Type B** = vertical jumper position (‖)

Set both jumpers to match the equipment you are connecting.

Fig. 2: Connection to MIDI IN minijack

2.3.3 Display Contrast Adjustment

The LCD contrast trimmer is accessible with the Raspberry Pi installed and the rear cover removed.

Use a small screwdriver to adjust the trimmer for the best display clarity.

Contrast is controlled by both:

- The hardware trimmer potentiometer
- Software contrast settings

For best results, install the software first, then adjust the trimmer so the software setting has a useful adjustment range.

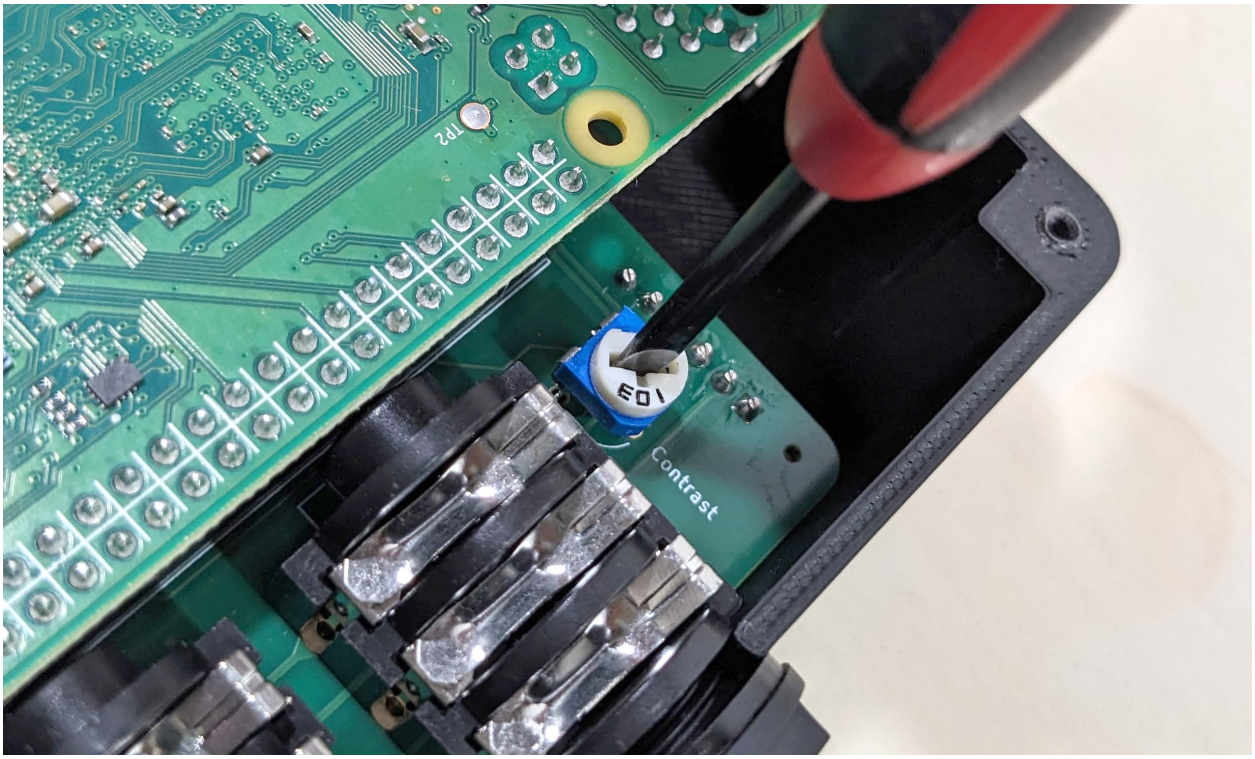


Fig. 3: Adjustment of hardware contrast

CHAPTER

THREE

ASSEMBLY

This section explains how to assemble a SquishBox from a kit, or from individually sourced parts after obtaining a fabricated PCB, printing an enclosure, and gathering the required through-hole components.

3.1 Components

In addition to the PCB (with surface-mount components pre-installed) and the 3D-printed PLA enclosure, a standard SquishBox kit includes:

A. 16x2 character LCD

B. 1x6 male header strips (2)

C. 2x20 female header

D. 10K trimmer potentiometer

E. 1/4" TRS audio jacks (2)

F. 2x2 male headers (2)

G. Jumper blocks (2)

H. Rotary pushbutton encoder + knob

Optional / Deluxe model:

J. Momentary footswitches (2)

K. 5mm LEDs (2)

3.2 Electronics

General soldering tips:

- Install components on the side of the PCB with the matching silkscreen outline.
- Seat components flush against the PCB, especially the rotary encoder, audio jacks, and 2x20 header, so everything aligns properly with the enclosure.
- Install the LCD last, as it covers several other solder points.

3.2.1 PCB Orientation

The surface-mount components are on the PCB "bottom" side, which faces the Raspberry Pi when installed.

Install these parts on the bottom side:

- Audio jacks

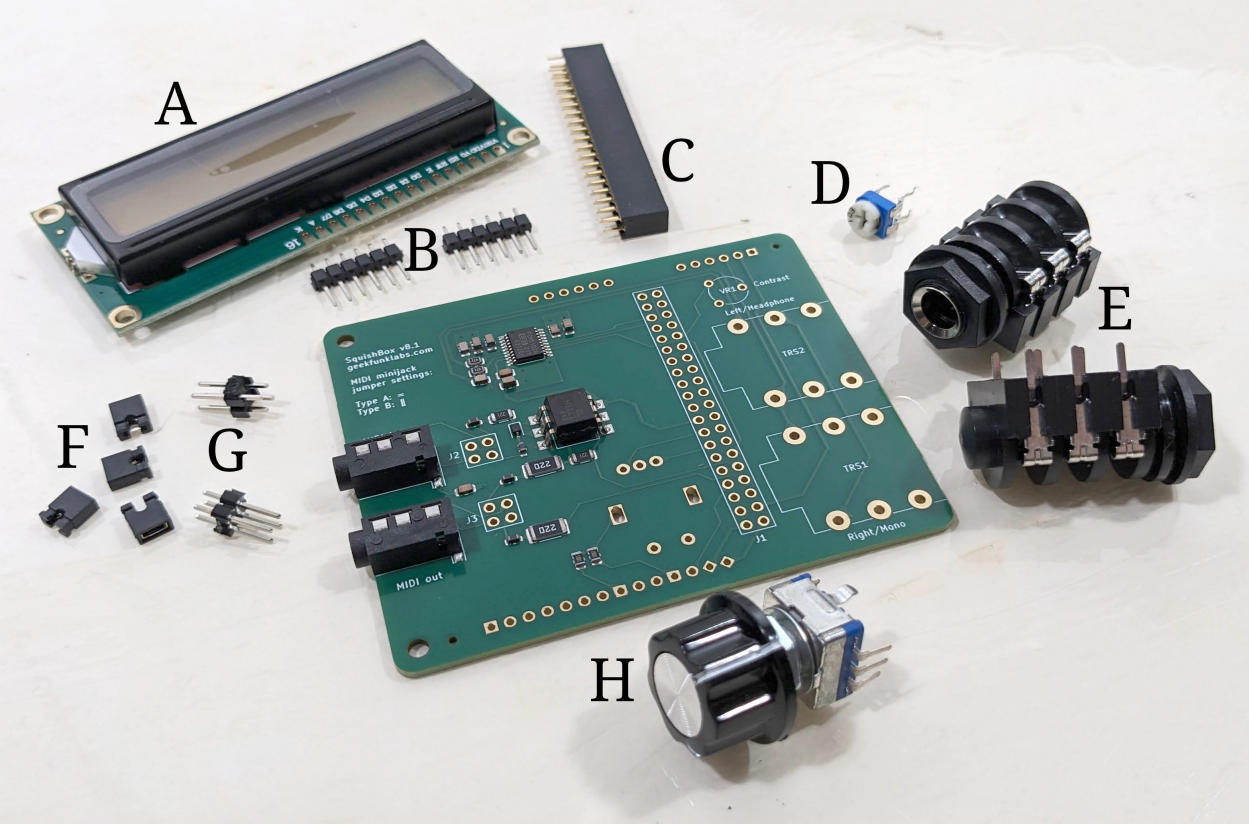


Fig. 1: Through-hole components for the SquishBox

- 2x2 headers
- 2x20 female header
- Trimmer potentiometer

Install these parts on the top side:

- Rotary encoder
- LCD module (last)

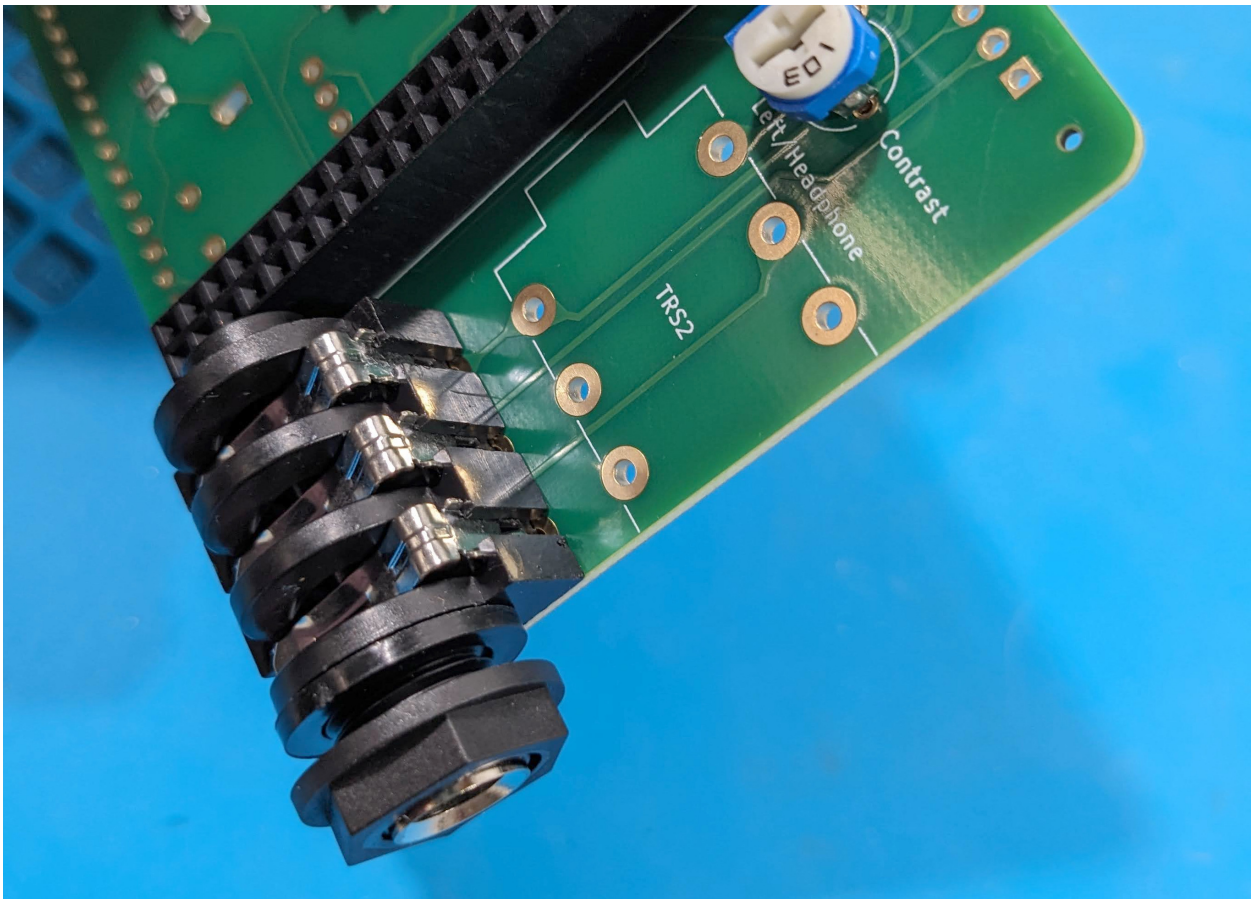


Fig. 2: Proper component mounting

3.2.2 MIDI Jumper Configuration

Headers J2 and J3 configure the MIDI TRS jacks for Type A or Type B wiring using jumper blocks.

As marked on the silkscreen:

- Type A: Horizontal connection (=)
- Type B: Vertical connection (‖)

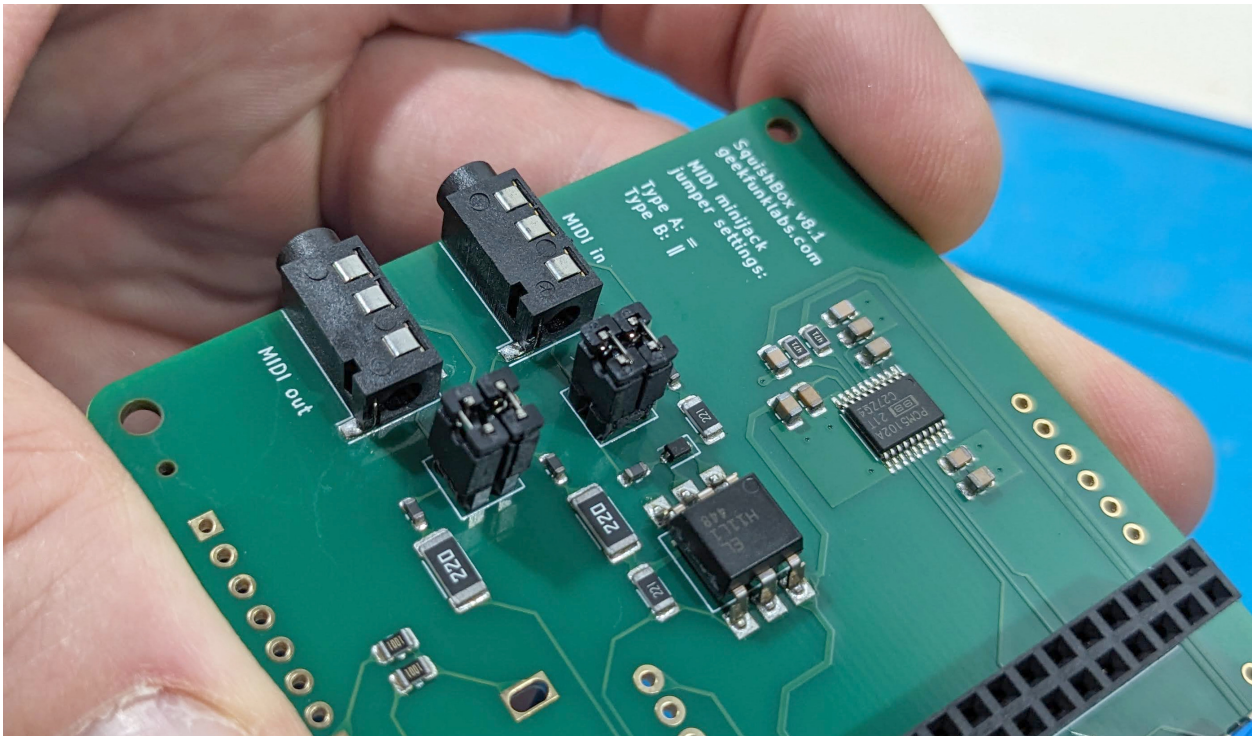


Fig. 3: MIDI minijack jumpers in type A configuration

3.2.3 LCD Header

The center four LCD pads (D0-D3) are unused.

Solder the 1x6 male header strips to the outer pads at each end of the LCD connector.

3.2.4 Deluxe Components

The Deluxe model adds two LEDs and two momentary footswitches.

LED Installation:

Insert the short leg (cathode / ground) of each LED into the square notched pads on header P1, positions 4 and 7. These pads connect to ground through 1K current-limiting resistors. Insert the long leg (anode / positive) into neighboring pads 3 and 8.

Leave enough lead length above the PCB so the LEDs can reach the enclosure openings before soldering.

Footswitch Wiring:

Strip and tin four short wires (about 3 cm each).

Wire the switches as follows:

- Connect one lug of each switch to P1 pads 12 and 13 (GPIO9 and GPIO10)
- Connect P1 pad 14 (GND) to the remaining lug of one switch
- Bridge that grounded lug to the remaining lug of the second switch

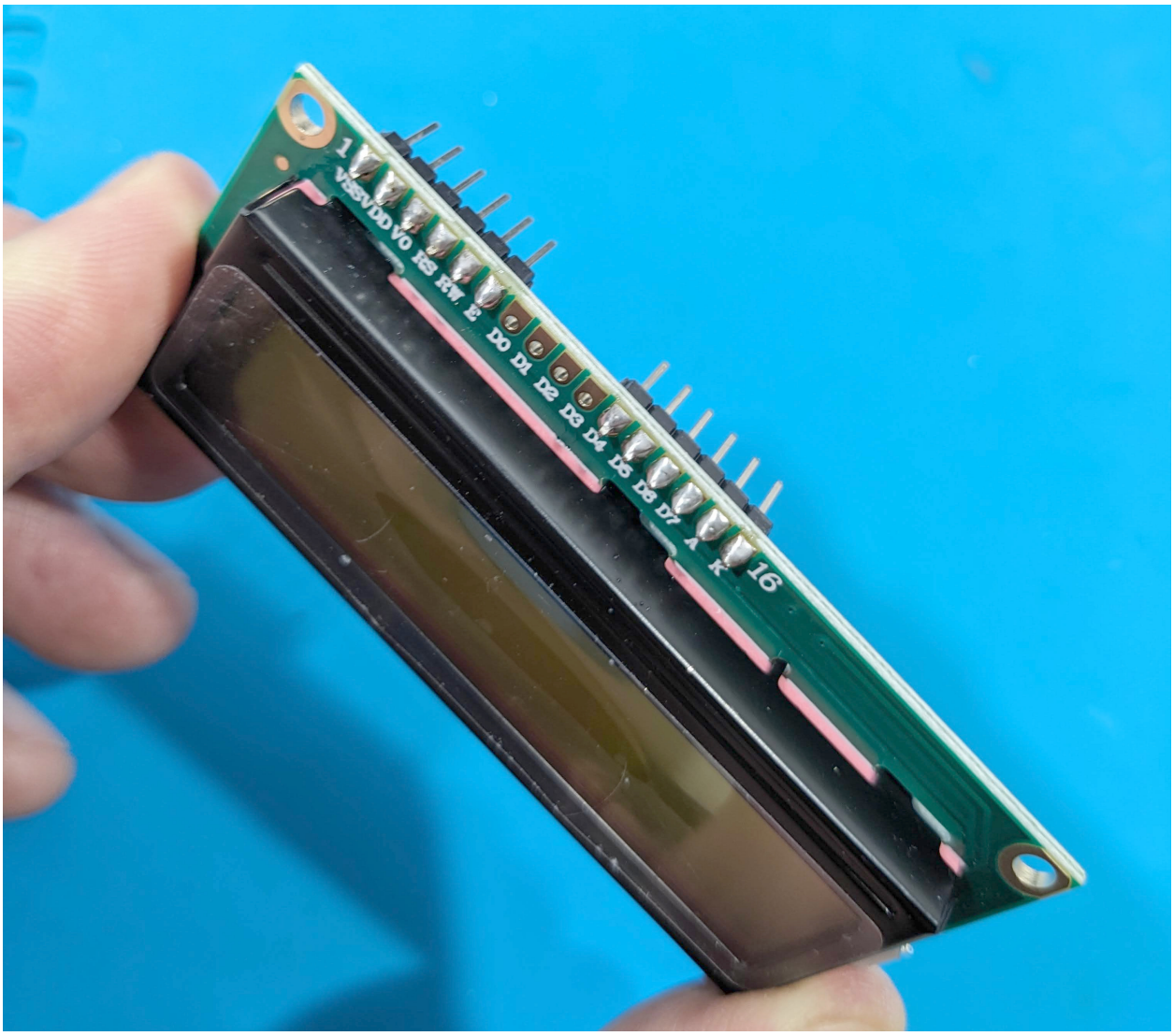


Fig. 4: Correct installation of pins to LCD connector

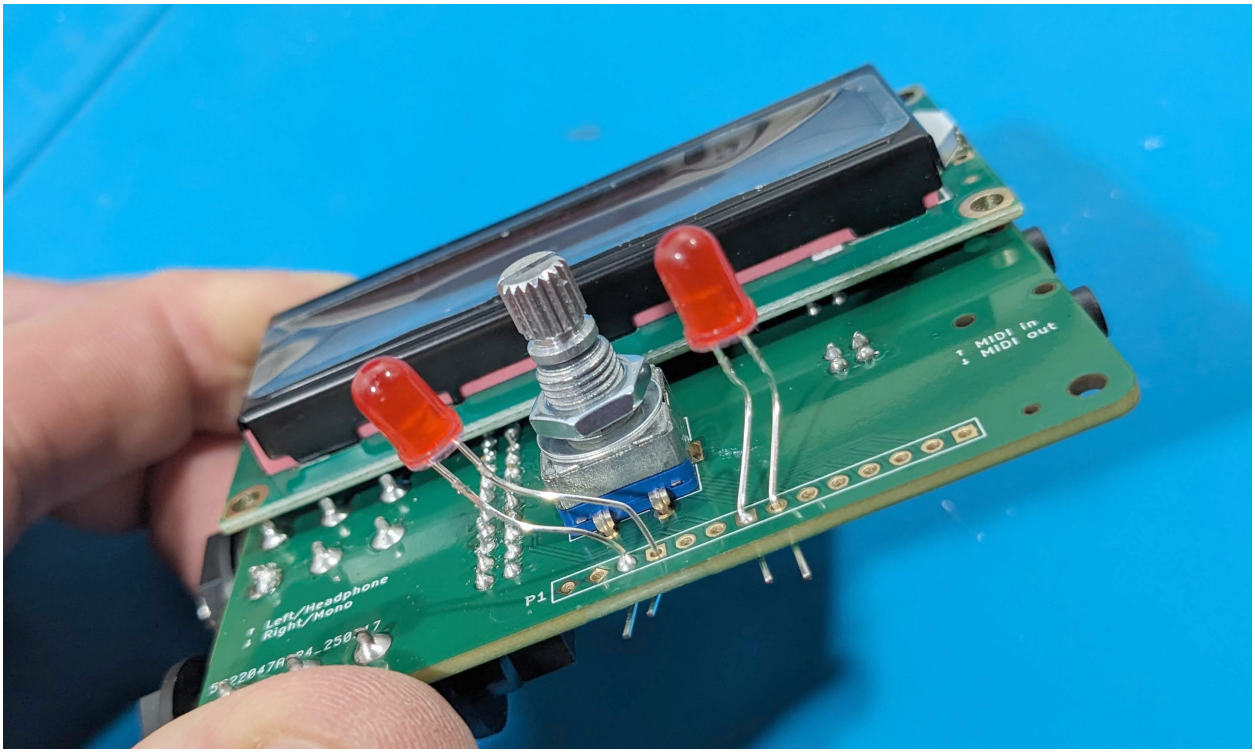


Fig. 5: Proper LED mounting

3.3 Enclosure

Align the rotary encoder shaft, LEDs (if installed), and audio jacks with the corresponding enclosure openings.

Press the PCB into place and secure it using the washers and nuts supplied with the rotary encoder and audio jacks. Mount stompswitches if used.

Install the Raspberry Pi by plugging its GPIO header into the 2x20 socket.

Secure the lid using the three included screws.

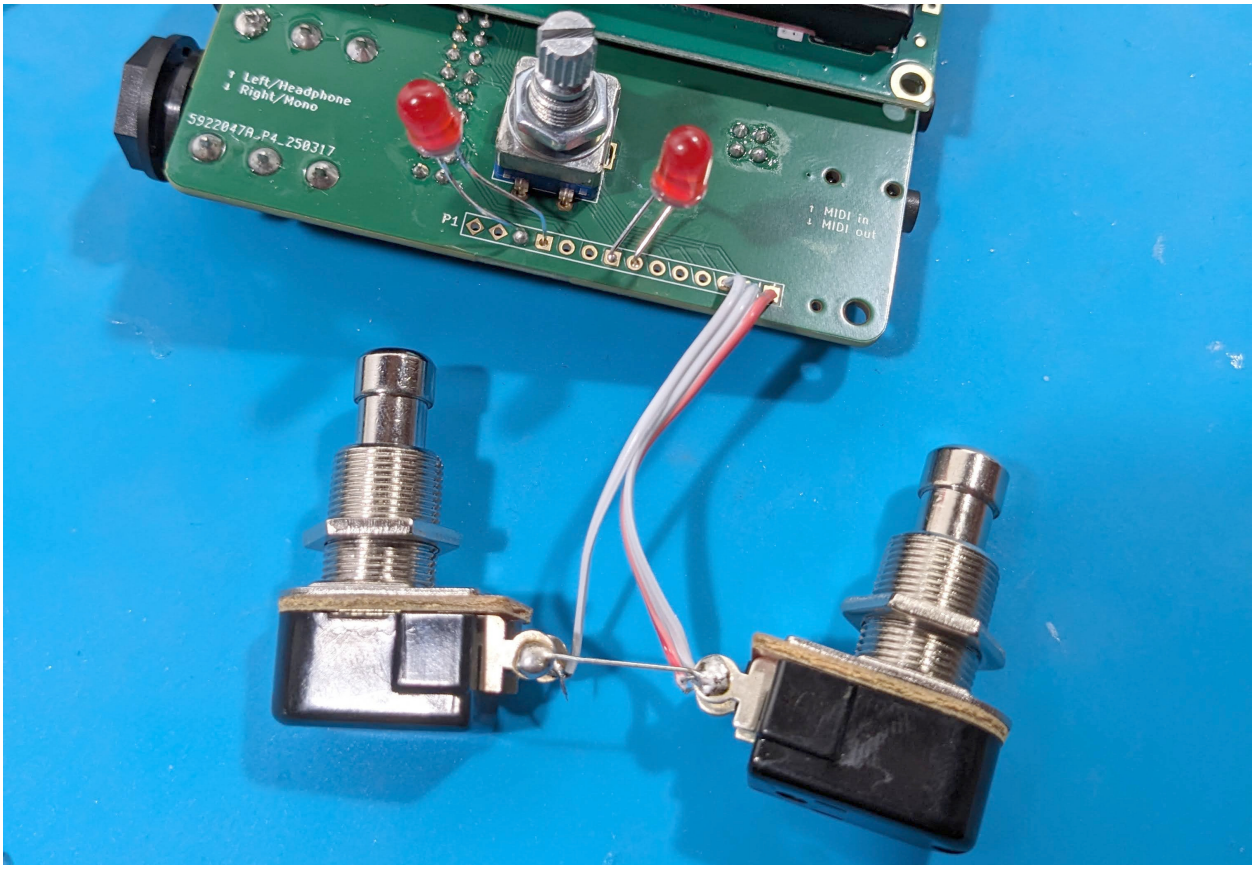


Fig. 6: Footswitch wiring

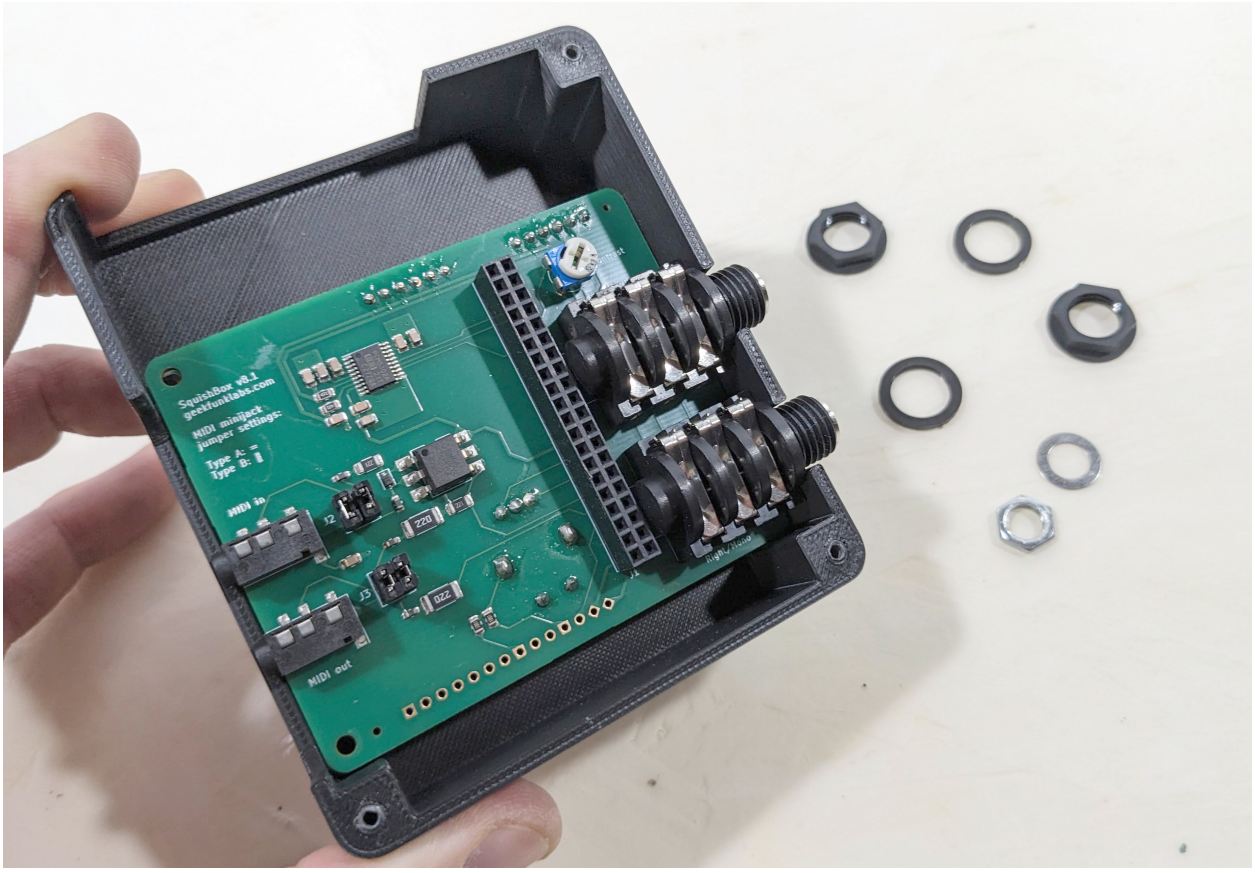


Fig. 7: PCB mounted in enclosure

Fig. 8: Raspberry Pi Installation

CHAPTER

FOUR

PROGRAMMING

This page is a practical primer for customizing/writing SquishBox apps.

Instead of dealing directly with GPIO timing, LCD protocols, encoder debouncing, or event queues, applications interact with a single high-level object

```
import squishbox
sb = squishbox.SquishBox()
```

From there you can:

- Write text to the LCD
- Read knob and button actions
- Present menus and prompts
- Edit text from the front panel
- Control LEDs and outputs
- Launch shell commands
- Build complete standalone hardware applications

4.1 Application Model

Most SquishBox programs follow a simple pattern:

1. Create a `SquishBox()` instance
2. Start the synth/audio process
3. Draw something on the LCD
4. Wait for user input
5. Respond to that input
6. Repeat until exit

Example:

```
import squishbox

sb = squishbox.SquishBox()
sb.lcd.clear()
sb.lcd.write("My Cool Synth", row=0)
```

(continues on next page)

(continued from previous page)

```
i = 0
synth = start_synth()
synth.select_patch(i)

while True:
    i, action = sb.menu_choose(synth.patchnames, i=i)

    if action == None:
        break
    else:
        synth.select_patch(i)
```

4.1.1 Configuration Files

The main SquishBox configuration file `squishboxconf.yaml` is intended to allow the user to change global behavior without modifying code.

The `controls` section defines named inputs such as buttons and encoders. Each item describes the hardware type, GPIO settings, and event bindings.

- `actions` Links events to SquishBox UI actions:
  - `inc`/`dec` change a value/option
  - `select` do/confirm
  - `back` cancel/return
- `messages` Emits MIDI messages in response to events. Message format is `<type>:<channel>:<number>:<value>`. Only control change messages (`ctrl`) are implemented.

```
controls:
  knob1:
    type: encoder
    pins: [22, 27]
    actions: {left: dec, right: inc}
  knob1_button:
    type: button
    pin: 17
    actions: {tap: select, hold: back}
  foot_left:
    type: button
    pin: 9
    messages: {in: ctrl:16:22:127, out: ctrl:16:22:0}
```

The main config file is loaded on import and stored in the `CONFIG` variable. The `load_config()` and `save_state()` functions can be used to manage config files for SquishBox apps.

```
from squishbox.config import load_config, save_state

myconfig = load_config("myappconf.yaml")

...
```

(continues on next page)

(continued from previous page)

```
myconfig["last_bank_path"] = current_bank
save_state("myappconf.yaml", myconfig)
```

Tokens ending with `_path` in config files are converted to `pathlib.Path` objects upon loading, and are serialized as POSIX strings when saving.

4.2 Controlling the LCD

The built-in LCD object is available as:

```
sb.lcd
```

Clear the display:

```
sb.lcd.clear()
```

Write text:

```
sb.lcd.write("Patch Loaded", row=0)
sb.lcd.write("Grand Piano", row=1, align="right")
```

Long text automatically scrolls when needed. Alignment is left by default. Non-scrolling text can be written on top of scrolling text to create partially-scrolling displays.

The `timeout=` option overlays text for a specified number of seconds:

```
sb.lcd.write("Saved", row=1, timeout=2)
```

This is useful for status messages.

4.2.1 Custom Characters

Characters beyond the standard "ASCII Printable" set can be defined as custom characters in the configuration file:

```
glyphs_5x8:
  wifi_on: |
    -XXX-
    X---X
    --X--
    -X-X-
    -----
    --X--
    -----
    -----
```

A maximum of 8 unique custom characters can be displayed at once, but an arbitrary number can be defined in the LCD object. They are displayed using element access:

```
sb.lcd.write("WiFi status: " + sb.lcd["wifi_on"], row=0)
```

4.3 User Interaction

The user interaction helpers use a blocking model - they don't return until a value is ready (unless `timeout=` is used). This makes it easier to understand program flow. They all have built-in idle loops that update scrolling elements of the LCD and provide CPU time for other processes such as synths/audio applications.

Several menu helper functions are provided to make it easy to create apps.

4.3.1 Choice Menu

```
i, option = sb.menu_choose(
    ["Piano", "Organ", "Synth"],
    row=1
)

if option:
    sb.lcd.write(option, row=0)
```

Returns:

- selected index
- selected item
- item is returned as `None` if canceled

4.3.2 Confirmation Prompt

```
if sb.menu_confirm("Delete file?"):
    delete_file()
```

Displays a toggling check/"X" at the end of text and returns True/False.

4.3.3 Text Entry

```
name = sb.menu_entertext("New Patch")
```

Useful for:

- patch names
- WiFi passwords
- filenames (with `charset=sb.lcd.fnchars()`)
- labels

Returns the entered text. Use `menu_confirm()` afterward to let the user confirm/cancel the input if desired.

4.3.4 File Browser

```
path = sb.menu_choosefile("/home/pi/patches", ext=[".yaml"])
```

This provides a simple two-line browser for selecting files. Returns a `Path()` object for the chosen file, or the last-browsed directory if canceled.

4.3.5 System Settings Menu

```
if sb.menu_systemsettings() == "shell":
    sys.exit()
```

This provides a unified system settings menu for LCD, WiFi, and MIDI settings. It also allows the user to shutdown/reboot the Pi, or exit the current program.

4.3.6 Direct Action Handling

For more complex user interfaces (e.g. vertically-scrollable menus, screens with active status indicators and/or custom actions), the `get_action()` function can be used to create interaction loops.

Incoming actions are stored in a queue, and `get_action()` retrieves them FIFO-style, blocking while the queue is empty (or until `timeout` is exceeded).

```
# display scrollable multi-line output

out = text.splitlines() # some multi-line text
irow, crow = 0, 0

while True:
    for i in range(irow, min(irow + ROWS, len(out))):
        sb.lcd.write(
            (out[i] if i < len(out) else "").ljust(COLS),
            row=i - irow
        )
    match sb.get_action():
        case "inc":
            crow += 1
            if crow == ROWS or crow == len(out):
                crow -= 1
                irow = min(irow + 1, len(out) - ROWS) % len(out)
        case "dec":
            crow -= 1
            if crow < 0:
                crow = 0
                irow = max(irow - 1, 0)
        case "select" | "back":
            break
```

The `add_action()` function can be used as a callback to send events to be picked up by `get_action()`. MIDI events are a common case.

```
def monitor_midi():
    while True:
        event = midi_input()
        sb.add_action(event)

...

while True:
    sb.lcd.write(patchnames[i], row=0)
    action = sb.get_action()
```

(continues on next page)

(continued from previous page)

```
    if action == "inc":
        i = (i + 1) % len(patchnames)
        select_patch(i)
    elif action == "dec":
        i = (i - 1) % len(patchnames)
        select_patch(i)
    elif isinstance(action, MidiEvent):
        sb.lcd.write(str(action), row=1, timeout=2)
```

4.3.7 Controlling Outputs

Outputs are defined in the configuration file:

```
outputs:
  led_blinker: {type: binary, pin: 23}
  led_fader: {type: pwm, pin: 4, level: 60}
```

Configured outputs are available through `sb.outputs`.

Example:

```
sb.outputs["led_blinker"].on()
sb.outputs["led_blinker"].off()
```

PWM outputs expose a `level` property representing duty cycle percentage.

```
sb.outputs["led_fader"].level = 75
```

4.4 Miscellaneous Tools

4.4.1 Running Shell Commands

The `shell_cmd()` method executes a string as a shell command and returns the output as an ASCII-encoded string.

```
result = sb.shell_cmd("hostname -I")
sb.lcd.write(result, row=1)
```

Useful for:

- audio tools
- system commands
- WiFi utilities
- file conversion
- launching synth engines

4.4.2 Long Running Tasks

Use the activity spinner while work is in progress:

```
with sb.lcd.activity("Loading..."):
    load_large_patch()
```

This gives visual feedback on the LCD while your task runs.

4.4.3 Error Handling

Unhandled exceptions are automatically displayed on the LCD before the application exits. To handle errors gracefully (i.e. without exiting):

```
try:
    load_patch(name)
except Exception as e:
    sb.display_error(e, "Load failed")
```

4.5 API Reference

The classes and functions below form the programming interface for the squishbox package.

4.5.1 SquishBox Module

class squishbox.squishbox.**SquishBox**

Bases: object

Interface to SquishBox hardware and UI.

Provides access to the LCD, controls (buttons/encoders), outputs, and a set of menu-driven interaction helpers. All hardware is initialized on first instantiation using values from CONFIG.

close()

Cleanly free the resources used by the SquishBox

menu_choose(*opts*, *row=1*, *align='right'*, *i=0*, *wrap=True*, *timeout=3.0*, *on_change=None*, *passthrough=None*)

Display a scrolling selection menu on the LCD.

The user navigates options via bound input actions ("inc", "dec", "select", "back"). The provided callback is invoked whenever the selection index changes.

Parameters

- **opts** (*list*) – Sequence of selectable items (displayed via str()).
- **row** (*int*) – LCD row to render the menu.
- **align** (*str*) – "left" or "right" text alignment.
- **i** (*int*) – Initial index.
- **wrap** (*bool*) – If True, selection wraps around at ends.
- **timeout** (*float*) – Seconds to wait for input (0 = wait indefinitely).
- **on_change** (*callable*) – Callback invoked as on_change(index) on selection change.
- **passthrough** (*tuple*) – actions/types to return immediately

Returns

(index, item) on selection (index, None) if canceled ("back") (-1, None) if timed out (-1, other) if an allowed action/type is passed

menu_confirm(*text=''*, *row=1*, *timeout=3.0*)

Display a yes/no confirmation prompt.

The user toggles between confirm (check) and cancel (cross) using "inc"/"dec", and confirms with "select".

Parameters

- **text** (*str*) – Prompt text.
- **row** (*int*) – LCD row to render the prompt.
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

Returns

True if confirmed False if explicitly rejected None if canceled ("back") or timed out

menu_entertext(*text=''*, *row=1*, *i=-1*, *timeout=0*, *charset=''*)

Interactive text entry using buttons/encoder or keyboard input.

Supports two cursor modes:

- "blink": move cursor position
- "line": modify character at cursor

Input sources:

- Encoder/buttons ("inc", "dec", "select", "back")
- Key events via keys_dispatch()

Parameters

- **text** (*str*) – Initial text buffer.
- **row** (*int*) – LCD row to render input.
- **i** (*int*) – Initial cursor index (negative values index from end).
- **timeout** (*float*) – Seconds to wait for input (0 = wait indefinitely).
- **charset** (*str*) – Allowed characters; defaults to LCD printable set.

Returns

Final edited string on completion.

menu_choosefile(*topdir*, *start=None*, *ext=None*, *row=0*, *timeout=0*)

Browse directories and select a file.

Displays a simple two-line file browser. Directories are shown with a trailing '/', and navigation includes a parent ("../") entry where applicable.

Parameters

- **topdir** (*Path*) – Root directory (user cannot navigate above this).
- **start** (*Path*) – Optional starting file or directory (relative to topdir).
- **ext** (*list*) – List of allowed file extensions (None = show all).
- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

Returns

selected file, or last directory if canceled

Return type
Path

`menu_lcdsettings`(*row=0*, *timeout=3.0*)

Adjust LCD contrast and backlight using slider UI.

Presents two sequential sliders. Changes are applied live and written back to the config file on exit.

Parameters

- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

`menu_midisettings`(*row=0*, *timeout=0*)

Manage MIDI input/output connections.

Lists available MIDI ports and allows toggling connections. Active connections are persisted to the config file and restored automatically on startup.

Parameters

- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

`property wifienabled`

Get/set WiFi radio state

`menu_wifisettings`(*row=0*, *timeout=0*)

Manage WiFi connectivity using nmcli.

Features:

- Enable/disable WiFi radio
- Scan for networks
- Connect/disconnect from access points
- Prompt for passwords when required

Displays connection status and IP address when available.

Parameters

- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

`menu_exit`(*row=0*, *timeout=3.0*)

System exit menu.

Allows the user to:

- Shutdown the system
- Reboot
- Exit to shell

Parameters

- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

Returns
"shell" if shell option is selected None otherwise (system may exit before returning)

`menu_systemsettings`(*row=0*, *timeout=3.0*)

Top-level system settings menu.

Provides access to LCD, MIDI, WiFi, and exit options.

Parameters

- **row** (*int*) – Starting LCD row (uses two rows).
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

Returns
"shell" if user selects exit-to-shell, otherwise None

Return type
str | None

`display_error`(*err*, *msg=''*, *row=0*)

Display an exception on the LCD and print traceback.

Formats the exception into a single-line message for the LCD, with file/line context shown on the second row. Full traceback is printed to stdout for debugging.

KeyboardInterrupt exits immediately.

Parameters

- **err** (*Exception*) – Exception instance
- **msg** (*str*) – Optional prefix message
- **row** (*int*) – Starting LCD row (uses two rows)

`add_action`(*name*)

Queue an input action.

Actions are typically generated by hardware event bindings.

`clear_actions`()

Remove all pending input actions.

`get_action`(*idle=0.01*, *timeout=0*)

Wait for and return the next input action.

Continuously updates the LCD while polling for actions.

Parameters

- **idle** (*float*) – Delay in seconds between polling iterations.
- **timeout** (*float*) – Seconds to wait (0 = wait indefinitely).

Returns
Next action from the queue, None if timed out

Return type
object

`static shell_cmd`(*cmd*, ***kwargs*)

Run a shell command and return its stdout.

Wrapper around subprocess.run with sensible defaults:

- check=True (raises on failure)
- captures stdout
- ASCII decoding
- strips trailing newline

Parameters

- **cmd** (*str | list*) – Command string (or list if shell=False)
- ****kwargs** (*dict*) – Passed through to subprocess.run

Returns
Command stdout as a stripped string

Return type
str

Raises
subprocess.CalledProcessError on failure (unless overridden) –

4.5.2 Hardware Module

SquishBox Raspberry Pi hardware interface.

Provides low-level access to GPIO-backed hardware components used by SquishBox, including buttons, rotary encoders, LCD display, and outputs.

Also implements event-driven input handling and a buffered LCD rendering system with support for scrolling, blinking, and custom glyphs.

Requires:

- gpiod

class squishbox.hardware.**Control**

Bases: `object`

Base class for input controls with event binding.

Provides a simple event → callback mapping. Subclasses trigger events (e.g. "tap", "left") which invoke the bound functions.

release()

bind(*event*, *func*)

Bind a callback function to an event.

Parameters

- **event** (*hashable*) – Event identifier
- **func** (*callable*) – function to invoke, or None to remove binding.

clear_binds()

Remove all event bindings.

class squishbox.hardware.**Button**(*pin*, *pull_up=True*)

Bases: *Control*

GPIO button with tap/hold detection.

Monitors a GPIO input line and emits events based on press duration.

Events:

- “down”: button pressed
- “up”: button released
- “in”/”out”: toggles on button press
- “tap”: short press
- “hold”: long press (duration >= CONFIG[“hold_time”])

`UP = 0`

`DOWN = 1`

`HELD = 2`

`class squishbox.hardware.Encoder(pin1, pin2, pull_up=True)`

Bases: *Control*

Quadrature rotary encoder.

Detects rotation direction using two GPIO inputs.

Events:

- “left”: counterclockwise rotation
- “right”: clockwise rotation

`class squishbox.hardware.Output(pin, on=False)`

Bases: `object`

Digital (on/off) GPIO output.

`release()`

`on()`

Set output to active (HIGH).

`off()`

Set output to inactive (LOW).

`class squishbox.hardware.PWMOutput(pin, freq=2000, level=0)`

Bases: `object`

Software PWM output using a background thread.

Generates a PWM signal by toggling a GPIO line at a fixed frequency. Duty cycle is controlled via the *level* attribute (0–100).

`level`

Duty cycle percentage (0-100)

Type
float

`release()`

`on()`

Set output to maximum.

off()

Set output to minimum.

class squishbox.hardware.**LCD_HD44780**(*regsel*, *enable*, *data*)

Bases: object

HD44780-compatible character LCD driver.

Provides buffered text rendering with support for:

- Static text
- Scrolling text (for long lines)
- Timed/blinking overlays
- Custom glyphs (up to 8 hardware slots)

Rendering is layered and only updates changed characters to minimize GPIO traffic.

glyph2char = (('backslash', '\\'), ('tilde', '~'))

release()

printable()

Provide full set of printable characters supported by the LCD.

fnchars()

Provide reduced character set suitable for filenames.

clear()

Clear the display and reset all rendering layers.

Also resets scrolling state, blinking timers, and cursor position.

write(*text*, *row*, *col=0*, *align=''*, *timeout=0*, *force=True*)

Write text to the LCD using layered rendering.

Behavior depends on text length and parameters:

- Long text is placed in scroll buffer
- timeout > 0 creates temporary (blinking) overlay
- Otherwise writes to static layer

Parameters

- **text** (*str*) – String to display.
- **row** (*int*) – Target row.
- **col** (*int*) – Starting column.
- **align** (*str*) – “left”, “right”, or default (no alignment).
- **timeout** (*float*) – Duration for temporary text (seconds).
- **force** (*bool*) – Overwrite existing timed text if True.

update()

Render buffered content to the LCD.

Handles:

- Scrolling updates
- Expiring timed/blinking text
- Layer compositing

Does nothing if activity spinner is active.

property cursor_pos

Cursor position as (row, col)

property cursor_mode

Cursor display mode.

Modes:

- "hide" – no cursor
- "blink" – blinking block cursor
- "line" - underline cursor

activity(*msg=None*)

Shows an animation while another process runs

Runs in a `with` statement. Displays a spinning character in the lower right corner of the LCD to give feedback while a long-running process completes.

Parameters
msg (*str*) – text to display

4.5.3 Config Module

squishbox.config.**load_config**(*name*, *default_cfg=None*)

Load configuration data

Loads config from YAML files. Creates a default config if target doesn't exist. Returns config as a dict after performing Path conversion on items with keys ending in `_path`.

Parameters

- **name** (*str | Path*) – config file relative to CONFIG_PATH or absolute
- **default_cfg** (*dict*) – overrides for initial config values

Returns
config data

Return type
dict

squishbox.config.**save_state**(*name*, *cfg*)

Write config data back to a file

Parameters

- **name** (*str | Path*) – config file relative to CONFIG_PATH or absolute
- **cfg** (*dict*) – the current config state

CHAPTER

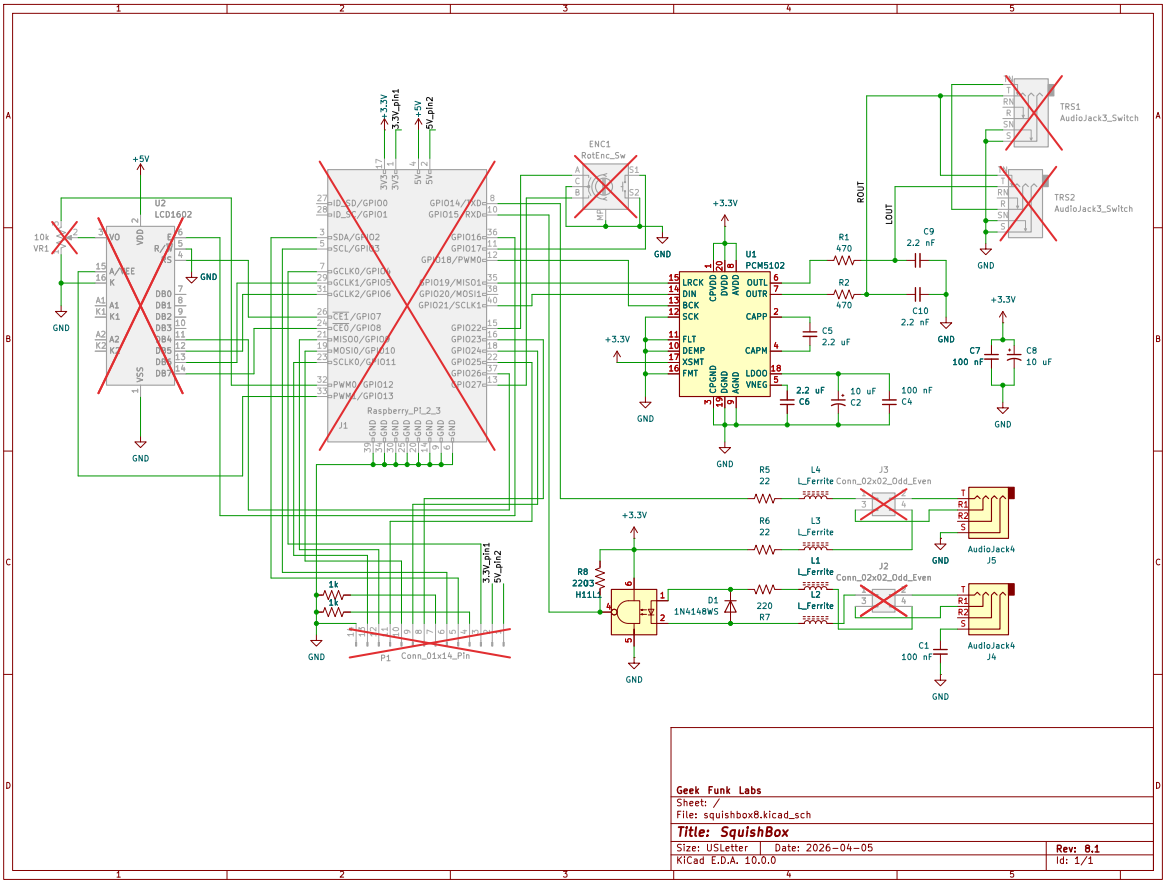
FIVE

TECHNICAL DOCS

This section collects miscellaneous hardware and software documentation for users modifying or servicing the SquishBox.

5.1 Schematic

The full schematic is available below. Through-hole components are crossed out to indicate they are sourced separately.



This Interactive Bill of Materials allows part identification, placement lookup, and assembly cross-referencing.

Open Interactive BOM

5.2 PCB

Download manufacturing files from https://github.com/GeekFunkLabs/squishbox/tree/main/hardware/pcb/fabrication.

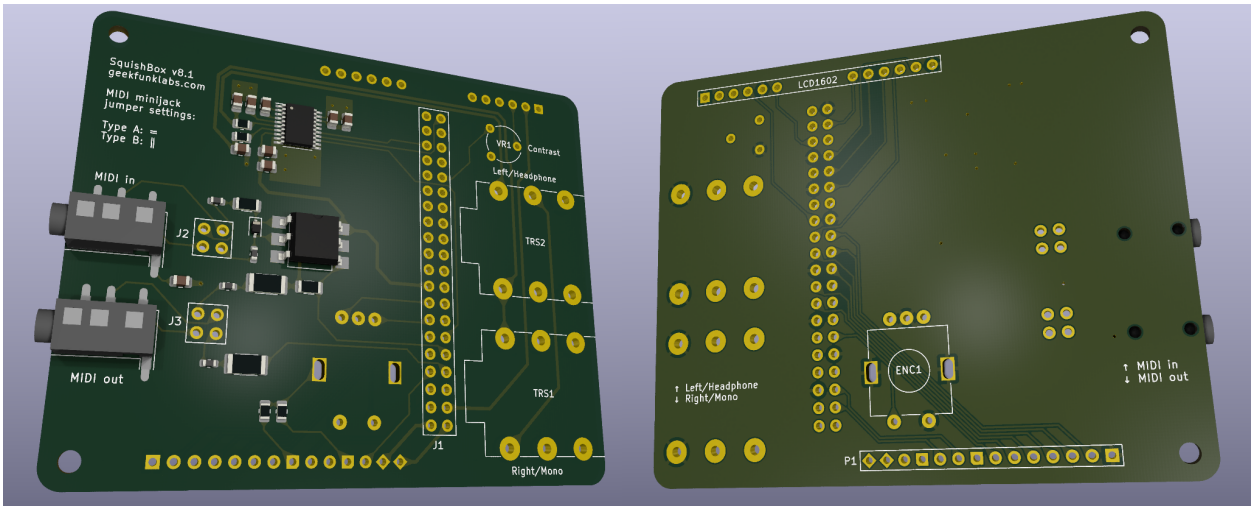


Fig. 1: 3D Render of SquishBox PCB with surface-mount components

5.2.1 P1 Header

Unused Raspberry Pi GPIO pins are broken out to header **P1** for switches, LEDs, sensors, and other add-ons.

Pins **4** and **7** include built-in 1k pull-down resistors to ground, allowing LEDs to be connected directly.

P1 Pin	Pad Shape	Function
1	diamond	5V
2	diamond	3.3V
3	circle	GPIO4
4	notched	GND via 1k
5	circle	GPIO2 (I2C SDA)
6	circle	GPIO3 (I2C SCL)
7	notched	GND via 1k
8	circle	GPIO23
9	circle	GPIO24
10	circle	GPIO10
11	circle	GPIO25
12	circle	GPIO9
13	circle	GPIO11
14	square	GND

5.3 System Setup

The installer script configures a complete SquishBox system by performing the following tasks:

- Installs system-level dependencies using Debian packages
- Creates a Python virtual environment and installs SquishBox and optional Python components

- Configures boot settings for supported hardware features such as audio devices and serial MIDI bridges
- Applies user-level configuration:
  - Updates group membership and permissions
  - Enables required system services
  - Creates convenience command aliases

5.3.1 Command Aliases

The installer script creates several convenience aliases for working with the SquishBox environment:

- `squishbox-python` - Runs Python inside the SquishBox virtual environment
- `squishbox-pip` - Installs or updates Python packages in the SquishBox virtual environment
- `squishbox-launcher` - Starts the SquishBox launcher in the current shell
- `squishbox-start` - Starts the SquishBox background service
- `squishbox-stop` - Stops the SquishBox background service
- `squishbox-status` - Displays the status of the SquishBox background service

5.3.2 Updates

Most application updates can be installed directly through *pip* inside the SquishBox virtual environment:

```
squishbox-pip install -U squishbox
```

To also upgrade optional components and their dependencies:

```
squishbox-pip install -U squishbox[full]
```

The installer script may be run again at any time to apply system-level changes, install new dependencies, or update system configuration.

PYTHON MODULE INDEX

S

INDEX

R

S

U

W